

Assignment 2 Parallel Programming

Student: Marcos Gómez Morena

Student ID: 2501102

Part 1: CUDA Warm-Up 1: SAXPY (20 pts)

Question 1. *What performance do you observe compared to the sequential CPU-based implementation of SAXPY (recall your results from saxpy on Program 5 from Assignment 1)?*

In the first assignment, with the CPU implementation of SAXPY I was able to achieve a bandwidth of ~7.6 GB/s. However, in the comparisons the parallel versions did not show speedup that was significant at all and sometimes it even resulted in slowdown, this was probably due to memory bandwidth saturation and task overhead.

In this assignment was very different, where the CUDA implementation shows a very high performance if we just consider the kernel execution time, achieving ~1100–1200 GB/s on the GPU. With this we are able to see a huge improvement over the CPU version due to the fact that we have a much higher memory bandwidth available on the GPU.

However, if we include the memory transfers between CPU and GPU, the total execution time increases quite a lot, reducing the effective bandwidth to ~6–8 GB/s, this being really similar to the CPU bandwidth, this indicates us that the performance is limited by data transfer rather than computation.

Overall, while the GPU greatly accelerates the computation itself, the benefit is reduced when considering data movement overhead.

Question 2. *Compare and explain the difference between the results provided by two sets of timers (timing only the kernel execution vs. timing the entire process of moving data to the GPU and back in addition to the kernel execution). Are the bandwidth values observed roughly consistent with the reported bandwidths available to the different components of the machine?*

There is a quite noticeable difference between the two timers. With the kernel-only timing measuring the execution of the SAXPY computation on the GPU, which is very fast (~1 ms), achieving bandwidths of ~1100–1200 GB/s. This results are consistent with the high bandwidth of the GPU's internal memory (HBM).

In the other hand, the total execution time also includes the memory transfers between the CPU and GPU, this increases in a significant way the runtime (~130–166 ms), with a resulting bandwidth of ~6–8 GB/s, which is much lower and consistent with PCIe bandwidth limitations.

Therefore, if we take into consideration the characteristics of the hardware the bandwidth values that we obtained are consistent: with a high bandwidth for on-device memory access and a much lower bandwidth for host-device communication.

Part 2: A Simple Circle Renderer (80 pts)

Machine Used

I run all the experiments for this project in Mahti using NVIDIA A100-SXM4-40GB GPU this GPU has the next specifications:

- 108 Streaming Multiprocessors (SMs)
- 40 GB HBM2 memory
- Compute Capability 8.0

For this assignment I was not able to use the GLUT display mode due to the fact that mahti required X11/GLUT libraries that were not available, so in order to completed I disable the mode. This will not affect how correct is the verifications or the performance measurements, because I used the `--chek` and `file-output` modes in order to perform the tests.

Performance Results

Scene	Render Time (ms)
rgb	0.03
rand10k	18.89
rand100k	188.48
pattern	1.81
snowsingle	52.42
biglittle	20.48
rand1M	1517.87
micro2M	2960.67

The precompiled reference binary `render_ref_x86` could not be executed on Mahti due to incompatible runtime dependencies (GLIBC, GLIBCXX, and libglut). Therefore, only the execution times of my CUDA implementation are reported.

I order to verify the correctness of the project I used the `--check` option, which compares the CUDA output against the sequential CPU reference renderer. Additionally I measured the CPU reference renderer using the `cpuref` mode.

For the `rand10k` scene, the CPU reference render time was 481.41 ms, while my CUDA implementation rendered the same scene in 18.89 ms, corresponding to an approximate 25× speedup.

```

[mgomezmo@g4102 render]$ mkdir -p logs
[mgomezmo@g4102 render]$ ./render -r cpuref -b 0:4 rgb -s 1024 -f logs/output
Rendering to 1024x1024 image
Loaded scene with 3 circles

Running benchmark, 4 frames, beginning at frame 0 ...
Dumping frames to logs/output_XXX.ppm
Wrote image file logs/output_0000.ppm
Wrote image file logs/output_0001.ppm
Wrote image file logs/output_0002.ppm
Wrote image file logs/output_0003.ppm
Clear:    50.5649 ms
Advance:  0.0014 ms
Render:   4.4337 ms
Total:    55.0000 ms
File IO:  67.8923 ms

Overall:  0.4916 sec (note units are seconds)
[mgomezmo@g4102 render]$ ./render -r cpuref -b 0:4 rand10k -s 1024 -f logs/output
Rendering to 1024x1024 image
Loaded scene with 10000 circles

Running benchmark, 4 frames, beginning at frame 0 ...
Dumping frames to logs/output_XXX.ppm
Wrote image file logs/output_0000.ppm
Wrote image file logs/output_0001.ppm
Wrote image file logs/output_0002.ppm
Wrote image file logs/output_0003.ppm
Clear:    50.3329 ms
Advance:  0.0014 ms
Render:   513.4517 ms
Total:    563.7860 ms
File IO:  70.8816 ms

Overall:  2.5387 sec (note units are seconds)

```

Initial Correct Implementation

In my first implementation, I decided to use a simple pixel-parallel strategy that solve in a pretty correct way the problem, in this implementation each CUDA thread is assigned to one pixel in the output image, iterating through all the circles in the input order, checking if the pixel center is inside each circle, and accumulating the resulting color in registers. After processing all the circles, the final color was written once to global memory.

This approach is guaranteed to be correct because:

1. Each pixel was updated by only one thread, eliminating race conditions.
2. Circles were processed sequentially in the original order, preserving the required blending order.
3. No locks or atomic operations were needed.

This solution has also some problems, being that every pixel test every circle, this results in a complexity of $O(\text{number of pixels} \times \text{number of circles})$, in very large scenes like rand1M and micro2M it became quite slow.

Final Optimized Implementation

I based this final implementation in the first one, but using a tiled rendering strategy with shared memory instead.

Work Decomposition

- Each CUDA block processes a tile of 16×16 pixels.
- Each block contains 256 threads.
- Circles are processed in batches of 128 circles.

Tile-Based Filtering

For each batch, threads load circle data into shared memory, later I used a conservative circle-box intersection test in order to determine if each circle intersects the current tile., doing that all threads in the block skip completely all the circles that do not overlap the tile.

Shared Memory Usage

I cached in shared memory this attributes of the circle:

- Position (x, y, z)
- Radius
- Color
- Activity flag indicating if the circle intersects the tile

All this is done in order to reduce redundant accesses to global memory.

Register Accumulation

Each thread accumulates the pixel color in registers and writes the final result to global memory only once after all relevant circles have been processed.

Synchronization

I used `__syncthreads()` in order to perform the synchronization, using two sync points for each circle batch:

1. After loading circle data into shared memory.
2. After all threads finish processing the batch before loading the next one.

Doing this there is no need for atomic operations or locks.

Parameter Tuning

I tested different configurations to see which one has the best performance:

- Tile sizes: 16×16, 32×8
- Batch sizes: 128, 256

After the tests I obtain the best overall performance with:

- Tile size: 16×16
- Batch size: 128

Being a configuration that provided the best balance across all benchmark scenes.

Optimization Process

For the development process I followed three stages:

1. **Starter code analysis** – The provided implementation assigned one thread per circle, which produced race conditions and incorrect blending order.
2. **Correct pixel-parallel implementation** – One thread per pixel ensured correctness.

3. **Tiled shared-memory optimization** – Reduced global memory traffic and avoided unnecessary circle tests using tile-level filtering.

In order to evaluate the performance I measured the render times across all benchmark scenes and compared the different tile and batch configurations.

Challenges Encountered

I encounter a few challenges during the developement of this project, being the main ones:

- Ensuring correct circle blending order.
- Eliminating race conditions without locks.
- Reducing the large computational cost of checking every circle for every pixel.
- Working around the absence of GLUT and X11 libraries on the Mahti cluster.(After asking the professor GLUT was able to work but I still needed the X11 :()

Conclusion

The final CUDA renderer is completely correct and way more efficient than the initial one. By combining tile-based work decomposition, shared memory caching, conservative tile-circle intersection tests, and register-based accumulation, the renderer is able to obtain a really good performance while preserving the strict correctness requirements of atomicity and circle ordering.

